

```
1 package nyp_proje;
2
3 /**
4  * Tüm araçların ortak üst sınıfı.
5  * Kamyon, Motorsiklet ve Drone bu sınıftan türetilir.
6  */
7 public abstract class Arac implements ILogger { //araç ortak özellik yani abstract class
8     // Araç bilgileri
9     protected String plaka;
10    protected double kapasite; // Maksimum taşıma kapasitesi
11    protected double yakit; // Mevcut yakıt miktarı
12    protected double hiz; // Araç hızı
13    protected Konum mevcutKonum; // Güncel konum
14
15    // Constructor
16    public Arac(String plaka, double kapasite, double yakit, double hiz, Konum
17        baslangicKonum) {
18        this.plaka = plaka;
19        this.kapasite = kapasite;
20        this.yakit = yakit;
21        this.hiz = hiz;
22        this.mevcutKonum = baslangicKonum;
23    }
24    // Getter metotları
25    public String getPlaka() {
26        return plaka;
27    }
28
29    public double getKapasite() {
30        return kapasite;
31    }
32
33    public double getYakit() {
34        return yakit;
35    }
36
37    public double getHiz() {
38        return hiz;
39    }
40
41    public Konum getMevcutKonum() {
42        return mevcutKonum;
43    }
44
45    /**
46     * Verilen mesafe kadar yakıt azaltır.
47     * Yakıt biterse otomatik dolum yapar.
48     */
49    protected void yakitTuket(double mesafe) {
50        yakit -= mesafe;
51
52        if (yakit <= 0) {
53            logHata(plaka + " yakıtı bitti! İkmal yapılıyor...");
54            yakitIkmal();
55        }
56    }
57
58    // Yakıt deposunu doldurur
59    public void yakitIkmal() {
60        yakit = 100;
```

```
61         logBilgi(plaka + " yakıt dolumu tamamlandı. Yakıt: " + yakit + " litre.");
62     }
63
64     // Aracı hedef konuma götürür
65     protected void konumaGit(Konum hedef) {
66
67         // İki konum arasındaki mesafeyi hesapla
68         double mesafe = mevcutKonum.mesafeHesapla(hedef);
69
70         // Mesafe kadar yakıt tüket
71         yakitTuket(mesafe);
72
73         // Yeni konumu güncelle
74         mevcutKonum = hedef;
75
76         // Bilgi mesajı yazdır
77         logBilgi(plaka + " konuma gitti: " + hedef
78             + " | Mesafe: " + String.format("%.2f", mesafe)); //Veriyi ekrana “düzenli
ve istenen biçimde” yazdırmak
79     }
80
81     /**
82     * Her araç teslimatı farklı yaptığı için
83     * alt sınıflar bu metodu override etmek zorundadır.
84     */
85     public abstract void teslimatYap(Paket paket); //araç “teslimat yapar” ama nasıl
yaptığı araç tipine göre değişir.
86
87     // Bilgi logu basar
88     @Override
89     public void logBilgi(String mesaj) {
90         System.out.println("[BİLGİ] [" + getClass().getSimpleName() + "] " + mesaj);
91     } //Bir sınıfın paketsiz (sadece sade) adını döndürür.
92
93     // Hata logu basar
94     @Override
95     public void logHata(String mesaj) {
96         System.out.println("[HATA] [" + getClass().getSimpleName() + "] " + mesaj);
97     }
98
99     // Nesne yazdırıldığında çalışır
100    @Override
101    public String toString() {
102        return getClass().getSimpleName() + "[" + plaka + " | Yakıt: "
103            + String.format("%.1f", yakit) + "L | Hız: " + hiz + "km/s]";
104    }
105 }
```